



Tema 1:
Análisis de Algoritmos (clase extra)

Estructura de Datos (2do. año)
 Carrera Analista de Sistemas/Licenciatura en Sistemas

Universidad Nacional de la Patagonia Austral
 Unidad Académica Río Gallegos

Sumatoria

- La sumatoria es una operación matemática que se emplea para calcular la suma de muchos o infinitos sumandos.
- La operación sumatoria se expresa con la letra griega sigma mayúscula E, y se representa así:

$$\sum_{i=1}^n x_i$$

- Donde:
 - x es la variable.
 - i es el valor inicial, llamado límite inferior.
 - n es el valor final, llamado límite superior.

Propiedades de la sumatoria

1. El número de sumandos y de términos de una sumatoria es igual al índice superior menos el índice inferior más una unidad.

$$\sum_{i=L_i}^{L_s} X_i = L_s - L_i + 1$$

2. La sumatoria de una constante es igual al producto del número de sumandos por la constante.
3. La sumatoria en el que el término general es una suma o una resta algebraica, se puede descomponer en sumatorias independientes (prop. distributiva).

$$\sum_{i=1}^5 (7i + 2) = \sum_{i=1}^5 7i + \sum_{i=1}^5 2$$

4. Una sumatoria cuyo índice inferior no es la unidad puede descomponerse como la sumatoria desde 1 a N menos la sumatoria de 1 a [S-1]. siendo S el limite inferior de la sumatoria general.
5. La sumatoria de una constante por un valor dependiente por el índice, es igual a la constante multiplicada por la sumatoria del índice.

Sumatorias notables

- Son sumatorias ya calculadas que permiten resolver problemas.

1. Los "n" primeros números naturales
2. Los "n" primeros números pares naturales
3. Los "n" primeros números impares naturales
4. Sumatoria de un grupo determinado por los límites

$$\sum_{i=L_i}^{L_s} i = \frac{(L_s - L_i + 1) * (L_s + L_i)}{2}$$

5. Suma de los primeros "n" cuadrados.

Aplicación de las Sumatorias al cálculo del $T(n)$

- Se tiene el siguiente fragmento de código:

Calcular $T(N)$

```

1) void unMetodo(){
2)   Scanner scanner = new Scanner(System.in); ← 2 OE
3)   int i, j, n, cont = 0; ← 1 OE
4)   n= scanner.nextInt(); ← 2 OE
5)   for (i = 1; i <= n; i++){
6)     for (j = 0; j < i; j++){
7)       System.out.print( cont++ ); ← 3 OE
8)       System.out.println(" "); ← 1 OE
9)     }
10) }
```

```

1) void unMetodo(){
2)   Scanner scanner = new Scanner(System.in); ← 2 OE
3)   int i, j, n, cont = 0; ← 1 OE
4)   n = scanner.nextInt(); ← 2 OE
5)   for (i = 1; i <= n; i++){ ← 1 OE
6)     for (j = 0; j < i; j++){ ← 1 OE
7)       System.out.print( cont++ ); ← 3 OE
8)     System.out.println(" "); ← 1 OE
9)   }
10) }

```

$$T(n) = 2 + 1 + 2 + (1 + 1 + \sum_{i=1}^N (1 + 1 + \sum_{j=0}^{i-1} (3 + 1 + 2))) + 1 + 1 + 2$$

$$T(n) = 5 + (6 + \sum_{i=1}^N (2 + \sum_{j=0}^{i-1} (6)))$$

$$T(n) = 5 + (6 + \sum_{i=1}^N (2 + (i - 1 - 0 + 1) * 6))$$

$$T(n) = 5 + (6 + \sum_{i=1}^N (2 + i * 6))$$

Se aplica propiedad de la sumatoria:

Distributiva

$$T(n) = 5 + (6 + \sum_{i=1}^N 2 + \sum_{j=0}^{i-1} (i * 6))$$

$$T(n) = 5 + (6 + \sum_{i=1}^N 2 + \sum_{j=0}^{i-1} (i * 6))$$

Se puede sacar las constantes de la sumatoria:

$$T(n) = 5 + (6 + \sum_{i=1}^N 2 + 6 * \sum_{j=0}^{i-1} i)$$

$$T(n) = 11 + ((n - 1 + 1) * 2 + 6) * \left(\frac{(n - 1 + 1) * (n + 1)}{2} \right)$$

$$T(n) = 11 + 2n + \left(6 * \frac{n^2 + n}{2} \right)$$

$$T(n) = 11 + 2n + \frac{6}{2} n^2 + \frac{6}{2} n$$

$$T(n) = 3n^2 + 5n + 11$$

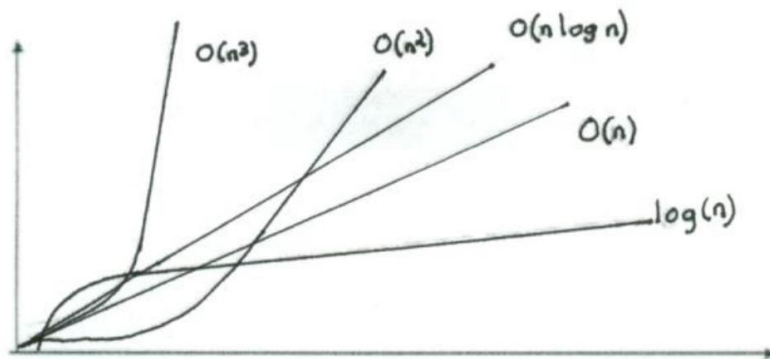
Complejidad cuadrática

Funciones de complejidad

- Ordenadas de mejor a la peor complejidad

- $O(1)$. Complejidad constante.
- $O(\log n)$. Complejidad logarítmica.
- $O(n)$. Complejidad lineal.
- $O(n \log n)$.
- $O(n^2)$. Complejidad cuadrática.
- $O(n^3)$. Complejidad cúbica.
- $O(n^k)$. Complejidad polinomial.
- $O(2^n)$. Complejidad exponencial.

Orden de Complejidad



Tipos de complejidad

- Existen diferentes tipos de complejidad, lo que se desea de un algoritmo es que su complejidad sea la menor posible.
- A continuación se presentan diferentes tipos de complejidad (las más comunes) ordenadas de menor a mayor.
 - $O(1)$: Complejidad constante
 - Es la más deseada. Por ejemplo, es la complejidad que se presenta en secuencias de instrucciones sin repeticiones o ciclos.
 - $O(\log n)$: Complejidad logarítmica
 - Puede presentarse en algoritmos iterativos y recursivos.
 - $O(n)$
 - Complejidad lineal. Es buena y bastante usual. Suele aparecer en un ciclo principal simple cuando la complejidad de las operaciones interiores es constante.

Tipos de complejidad

- $O(n \log n)$ Complejidad $n \cdot \log n$
 - Se considera buena. Aparece en algunos algoritmos de ordenamiento.
- $O(n^2)$ Complejidad cuadrática
 - Aparece en ciclos anidados, por ejemplo ordenación por burbuja. También en algunas recursiones dobles.
- $O(n^3)$ Complejidad cúbica
 - En ciclos o en algunas recursiones triples. El algoritmo se vuelve lento para valores grandes de n .
- $O(n^k)$ Complejidad polinomial
 - Para $(k \in \mathbb{N}, k > 3)$ mientras más crece k , el algoritmo se vuelve más lento.
- $O(C^n)$ Complejidad exponencial
 - Cuando n es grande, la solución de estos algoritmos es prohibitivamente costosa. Por ejemplo, problemas de explosión combinatoria.

Complejidad en algoritmos recursivos

- Anteriormente se explicó como determinar el tiempo de ejecución de un algoritmo a partir del computo de sus operaciones elementales
- En general, este computo se reduce a un ejercicio de calculo.
- Sin embargo, para los algoritmos recursivos se encuentran con una dificultad añadida
 - la función que establece su tiempo de ejecución viene dada por una ecuación de recurrencia, es decir $T(n) = E(n)$, donde en la expresión E aparece la propia función T.
- Resolver este tipo de ecuaciones consiste en encontrar una expresión no recursiva de T, y por lo general no es algo fácil.

Algoritmo recursivo

- Un algoritmo recursivo consiste en obtener la solución total de un problema a partir de la solución de casos más sencillos del mismo problema.
- Estos casos se resuelven invocando al mismo algoritmo, el problema se simplifica hasta llegar al caso más sencillo de todos llamado caso base.
- Para analizar la complejidad de un algoritmo recursivo se utilizan las ecuaciones de recurrencia
 - permiten indicar el tiempo de ejecución para los distintos casos de un algoritmo recursivo (caso base y caso general).

ASPECTO GENERAL DE UN ALGORITMO RECURSIVO

```

algoritmo recursivo (caso)
inicio
  if caso == casobase
    regresa solucionpordefinicion
  else
    ...
    s1 = recursivo (casoreducido1)
    s2 = recursivo (casoreducido2)
    ...
    regresa soluciondelcaso (s1, s2,...)
  
```

¿CÓMO CALCULAR LAS ECUACIONES DE RECURRENCIA?

- Una vez que se tienen las ecuaciones de recurrencia, es posible obtener la expresión de la función de coste, en la que se tienen las siguientes partes fundamentales:
 - Casobase: valor (n) que provoca el fin de la recursión.
 - a: indica el número de llamadas recursivas que se generan a partir de cada llamada recursiva.
 - op c: la forma en la que disminuye el tamaño del problema n de llamada en llamada, normalmente la reducción es del tipo:
 - n - c: Siendo c una constante tal que $c \geq 1$ (progresión aritmética)
 - n / c: Siendo c una constante tal que $c > 1$ (progresión geométrica)
 - El coste iterativo b indica el coste de las OE realizadas por el método excluyendo la invocación de las llamadas recursivas.
- En términos generales:

$$T(n) = \begin{cases} k & T(\text{CasoBase}) \\ a * T(n \text{ op } c) + b & T(\text{CasoGeneral}) \end{cases}$$

Ejemplo: factorial de un numero natural

- Programa

```
int factorial(int n){
    if(n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
```

$$T(n) = \begin{cases} 2 & \text{Si } n < 1 \\ T(n-1) + 4 & \text{Si } n > 0 \end{cases}$$

- Análisis:

- Tamaño:** el valor del parametro n.
- Caso base:** cuando el valor de n es igual a 0, ya que terminaría las llamadas recursivas, entonces el T(0) es igual a 2, 1 por la comparación en el **if** y 1 por el **return**.
- Caso general:** cuando el valor de n es mayor a 0,
- entonces el T(n) esta dado por: 4, 1 por la comparación en el **if**, 1 por el producto, 1 por la resta y 1 por el **return** (las llamadas a las funciones recursivas no se tienen en cuenta), a esto hay que sumarle que la funcion se llama asi misma con la reducción del tamaño del problema en una unidad, es decir, T(n-1)
- en resumen, el $T(n) = 4 + T(n - 1)$.

- Entonces se obtiene la ecuación de recurrencia

EN EL EJERCICIO ANTERIOR: CUANTO VALE EL T(6)?

- Esta pregunta es una pregunta recurrente

- ya que para saber el T(6), se tiene que saber el T(5), y para saber el T(5), es necesario saber el T(4) y así hasta poder llegar al T(0) que es nuestro caso base.

- Entonces, para el caso general: la recursión de las llamadas a la función

$$\begin{aligned} T(n) &= T(n-1) + 4 \\ T(6) &= T(6-1) + 4 = T(5) + 4 \\ T(6) &= (T(5-1) + 4) + 4 = T(4) + 8 \\ &\vdots \\ T(6) &= (T(1-1) + 4) + 20 = T(0) + 24 \end{aligned}$$

- Por definición $T(0) = 2$, entonces:

- $T(6) = 2 + 24 = 26$
- $O(1)$ = Complejidad constante

SI AHORA QUIERO SABER CUANTO VALE EL $T(15)$?

- Podríamos seguir resolviendo como en el ejemplo anterior, reemplazando los valores de $T(n)$ hasta llegar al caso que se requiere, es decir, $T(15)$, $T(14)$... $T(0)$
- Pero antes se podría analizar:
 - El valor de diferencia entre cada $T(n)$ y $T(n - 1)$, siendo $n \geq 1$, es de 4.
 - El valor del $T(0) = 2$ ya que es nuestro caso base.
- Entonces se concluye que por cada $n \geq 1$ se agrega un valor fijo 4, al valor fijo 2, entonces, matemáticamente se establece la siguiente hipótesis:
 - $T(n) = 2 + 4 * n$
 - $O(n) = \text{Complejidad Lineal}$
- De esta manera, cada vez que nuestro valor de n sea mayor o igual a 1, se multiplica n por 4, y se le suma el valor del caso base 2.

VERIFICACION DE LA HIPOTESIS ANTERIOR

- Sabiendo que: $T(N) = 2 + 4 * N$, entonces:
 - $T(0) = 2 + 4 * 0 = 2$
 - $T(1) = 2 + 4 * 1 = 6$
 - ...
 - $T(6) = 2 + 4 * 6 = 2 + 24 = 26$
- Aquí se evidencia que la formula general del $T(n)$ es correcta, de esta manera:
 - $T(15) = 2 + 4 * 15 = 62$
 - $O(1) = \text{Complejidad Constante}$